

Universidad Tecnológica de Panamá
Facultad de Sistemas Computacionales
Asignatura: Programación II
Taller Práctico1
Profesor: Napoleón Ibarra
Valor: 100 puntos

Estudiante: Magdalena Gonzalez

Cédula: 4-819-1590

```
import customtkinter as ctk
from tkinter import messagebox, filedialog
import json
import os
import re
import base64
```

```
try:
    from cryptography.fernet import Fernet
    from cryptography.hazmat.primitives.asymmetric import rsa, padding
    from cryptography.hazmat.primitives import serialization, hashes
    CRYPTO_OK = True
except ImportError:
    CRYPTO_OK = False
```

```
try:
    from PIL import Image
    PIL_OK = True
except ImportError:
    PIL_OK = False
```

```
ctk.set_appearance_mode("dark")
ctk.set_default_color_theme("blue")
```

```
BG_DARK   = "#0f0f1a"
BG_PANEL  = "#1a1a2e"
BG_CARD   = "#16213e"
SIDEBAR_BG = "#0d0d1f"
ACCENT    = "#4e73df"
SUCCESS   = "#1cc88a"
DANGER    = "#e74a3b"
WARNING   = "#f6c23e"
TEXT_WHITE = "#e8e8f0"
TEXT_DIM  = "#7a7a9a"
BORDER    = "#2a2a4a"
```

```
APP_TITLE   = "Sistema Académico Tec"
DATA_FILE   = "academic_data.json"
LOGO_PATH   = r"C:\Users\Sari8\PyCharmMiscProject\images\uni.png"
CANDADO_PATH = r"C:\Users\Sari8\PyCharmMiscProject\images\candado.png"
```

```
LLAVE_PATH = r"C:\Users\Sari8\PyCharmMiscProject\images\llave.png"
USUARIO_PATH = r"C:\Users\Sari8\PyCharmMiscProject\images\user.png"
```

```
TIPOS_EVALUACION = ["Tareas", "Laboratorios", "Parciales", "Portafolio",
                    "Asistencia", "Proyecto", "Semestral"]
```

```
LIM = {
    "nombre_asig": 60,
    "codigo_asig": 10,
    "nombre_grupo": 30,
    "nombre_cat": 40,
    "pct": 5,
    "nombre_est": 60,
    "cedula": 15,
    "nombre_nota": 40,
    "valor_nota": 5,
}
```

```
ALNUM_GUION =
"ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789-_"
CEDULA_CHARS =
"ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789-"
NOMBRE_CHARS = (
    "ABCDEFGHIJKLMNPOQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"
    "ÁÉÍÓÚáéíóúÑñÜü "
    ".,'"
)
```

```
RE_CEDULA_NACIONAL = re.compile(r'^\d{1,2}-\d{1,4}-\d{1,5}$')
RE_CEDULA_EXTRANJERO = re.compile(r'^(E|N|PE|AV|PI)-\d{1,2}-\d{1,6}$', re.IGNORECASE)
```

```
def _cedula_valida(cedula):
    c = cedula.strip()
    if RE_CEDULA_NACIONAL.match(c):
        return True
    if RE_CEDULA_EXTRANJERO.match(c):
        return True
    return False
```

```
def _nombre_valido(nombre):
    if not nombre or not nombre.strip():
        return False
    if "@" in nombre:
        return False
    if not all(c in NOMBRE_CHARS for c in nombre):
        return False
    if nombre.strip().replace(" ", "").replace(".", "").replace(",", "").replace("'", "").replace("-", "") == "":
        return False
    return True
```

```
class DataModel:
    def __init__(self, filename=DATA_FILE):
        self.filename = filename
```

```

self.asignaturas = []
self.grupos = []
self.estudiantes = []
self.categorias = []
self.notas = {}
self.load()

def _new_id(self, items):
    return max((i["id"] for i in items), default=0) + 1

def _nota_valor(self, n):
    return n["valor"] if isinstance(n, dict) else float(n)

def _nota_nombre(self, n, idx):
    return (n.get("nombre") or f"Nota {idx+1}") if isinstance(n, dict) else f"Nota {idx+1}"

def _parse_pct_tipo(self, porcentaje, tipo):
    try: pct = float(porcentaje)
    except Exception: return None, "Porcentaje inválido."
    if pct <= 0 or pct > 100: return None, "Porcentaje entre 1 y 100."
    if tipo == "Semestral" and pct < 30: return None, "Semestral debe ser mínimo 30%."
    return pct, None

def _parse_nota(self, nota):
    try: v = float(nota)
    except Exception: return None, "Nota inválida."
    if not 0 <= v <= 100: return None, "La nota debe estar entre 0 y 100."
    return v, None

def save(self):
    try:
        with open(self.filename, "w", encoding="utf-8") as f:
            json.dump({k: getattr(self, k) for k in
                        ("asignaturas", "grupos", "estudiantes", "categorias", "notas")},
                      f, indent=2, ensure_ascii=False)
    except Exception:
        pass

def load(self):
    if not os.path.exists(self.filename):
        return
    try:
        with open(self.filename, "r", encoding="utf-8") as f:
            d = json.load(f)
            self.asignaturas = d.get("asignaturas", [])
            self.grupos = d.get("grupos", [])
            self.estudiantes = d.get("estudiantes", [])
            self.notas = d.get("notas", {})
            self.categorias = [c for c in d.get("categorias", []) if "asig_id" in c]
    except Exception:
        pass

def get_asignatura(self, aid): return next((a for a in self.asignaturas if a["id"] == aid), None)
def get_grupo(self, gid): return next((g for g in self.grupos if g["id"] == gid), None)
def get_estudiante(self, eid): return next((e for e in self.estudiantes if e["id"] == eid), None)
def get_categoria(self, cid): return next((c for c in self.categorias if c["id"] == cid), None)

```

```

def grupos_de_asignatura(self, aid): return [g for g in self.grupos if g["asig_id"] == aid]
def estudiantes_de_asignatura(self, aid): return [e for e in self.estudiantes if e["asig_id"] == aid]
def categorias_de_asignatura(self, aid): return [c for c in self.categorias if c["asig_id"] == aid]

def total_pct(self, asig_id):
    return sum(c["porcentaje"] for c in self.categorias_de_asignatura(asig_id))

def puede_calcular_final(self, est_id):
    est = self.get_estudiante(est_id)
    if not est: return False, "Estudiante no encontrado"
    cats = self.categorias_de_asignatura(est["asig_id"])
    if not cats: return False, "Sin categorías definidas"
    if self.total_pct(est["asig_id"]) != 100:
        return False, f"Porcentajes suman {self.total_pct(est['asig_id'])}%"
    if not any(c["tipo"] == "Semestral" for c in cats):
        return False, "Falta categoría Semestral"
    for cat in cats:
        if not self.notas.get(str(est_id), {}).get(str(cat["id"]), []):
            return False, f"Sin notas en '{cat['nombre']}'"
    return True, "OK"

def nota_final_est(self, est_id):
    ok, _ = self.puede_calcular_final(est_id)
    if not ok: return None
    est = self.get_estudiante(est_id)
    cats = self.categorias_de_asignatura(est["asig_id"])
    total = sum(
        (sum(self._nota_valor(n) for n in self.notas[str(est_id)][str(cat["id"])]) /
         len(self.notas[str(est_id)][str(cat["id"])])) * (cat["porcentaje"] / 100)
        for cat in cats
    )
    return round(total, 2)

def add_asignatura(self, nombre, codigo):
    if not nombre or not codigo: return False, "Completa nombre y código."
    if any(a["codigo"].lower() == codigo.lower() for a in self.asignaturas):
        return False, "Ya existe ese código."
    self.asignaturas.append({"id": self._new_id(self.asignaturas), "nombre": nombre, "codigo":
codigo})
    self.save(); return True, "Asignatura agregada."

def update_asignatura(self, asig_id, nombre, codigo):
    if not nombre or not codigo: return False, "Completa nombre y código."
    if any(a["codigo"].lower() == codigo.lower() and a["id"] != asig_id for a in self.asignaturas):
        return False, "Ya existe ese código en otra asignatura."
    a = self.get_asignatura(asig_id)
    if not a: return False, "No encontrada."
    a["nombre"] = nombre; a["codigo"] = codigo
    self.save(); return True, "Asignatura actualizada."

def delete_asignatura(self, asig_id):
    self.asignaturas = [a for a in self.asignaturas if a["id"] != asig_id]
    gids = [g["id"] for g in self.grupos if g["asig_id"] == asig_id]
    eids = [e["id"] for e in self.estudiantes if e["grupo_id"] in gids]
    cids = [c["id"] for c in self.categorias if c["asig_id"] == asig_id]

```

```

self.grupos = [g for g in self.grupos if g["asig_id"] != asig_id]
self.estudiantes = [e for e in self.estudiantes if e["grupo_id"] not in gids]
self.categorias = [c for c in self.categorias if c["asig_id"] != asig_id]
for eid in eids: self.notas.pop(str(eid), None)
for cid in cids:
    for ek in list(self.notas): self.notas[ek].pop(str(cid), None)
self.save()

def add_grupo(self, asig_id, nombre, modalidad):
    if not nombre: return False, "Completa el nombre del grupo."
    if not self.get_asignatura(asig_id): return False, "Asignatura inválida."
    self.grupos.append({"id": self._new_id(self.grupos), "asig_id": asig_id,
                        "nombre": nombre, "modalidad": modalidad})
    self.save(); return True, "Grupo agregado."

def update_grupo(self, grupo_id, nombre, modalidad):
    g = self.get_grupo(grupo_id)
    if not g: return False, "Grupo no encontrado."
    if not nombre: return False, "El nombre no puede estar vacío."
    g["nombre"] = nombre; g["modalidad"] = modalidad
    self.save(); return True, "Grupo actualizado."

def delete_grupo(self, grupo_id):
    eids = [e["id"] for e in self.estudiantes if e["grupo_id"] == grupo_id]
    self.grupos = [g for g in self.grupos if g["id"] != grupo_id]
    self.estudiantes = [e for e in self.estudiantes if e["grupo_id"] != grupo_id]
    for eid in eids: self.notas.pop(str(eid), None)
    self.save()

def add_categoria(self, asig_id, nombre, porcentaje, tipo):
    if not self.get_asignatura(asig_id): return False, "Selecciona una asignatura válida."
    if not nombre: return False, "El nombre no puede estar vacío."
    pct, err = self._parse_pct_tipo(porcentaje, tipo)
    if err: return False, err
    cats = self.categorias_de_asignatura(asig_id)
    if any(c["tipo"] == tipo for c in cats):
        return False, f"Ya existe una categoría '{tipo}' en esta asignatura."
    if self.total_pct(asig_id) + pct > 100:
        return False, f"Excede 100%. Disponible: {100 - self.total_pct(asig_id)}%"
    self.categorias.append({"id": self._new_id(self.categorias), "asig_id": asig_id,
                           "nombre": nombre, "porcentaje": pct,
                           "tipo": tipo, "es_final": tipo == "Semestral"})
    self.save(); return True, "Categoría agregada."

def update_categoria(self, cat_id, nombre, porcentaje, tipo):
    cat = self.get_categoria(cat_id)
    if not cat: return False, "No encontrada."
    if not nombre: return False, "El nombre no puede estar vacío."
    pct, err = self._parse_pct_tipo(porcentaje, tipo)
    if err: return False, err
    cats = self.categorias_de_asignatura(cat["asig_id"])
    if any(c["tipo"] == tipo and c["id"] != cat_id for c in cats):
        return False, f"Ya existe una categoría '{tipo}' en esta asignatura."
    total_sin = self.total_pct(cat["asig_id"]) - cat["porcentaje"]
    if total_sin + pct > 100:
        return False, f"Excede 100%. Disponible: {100 - total_sin}%"

```

```

cat.update({"nombre": nombre, "porcentaje": pct,
          "tipo": tipo, "es_final": tipo == "Semestral"})
self.save(); return True, "Categoría actualizada."

def delete_categoria(self, cat_id):
    self.categorias = [c for c in self.categorias if c["id"] != cat_id]
    for ek in list(self.notas): self.notas[ek].pop(str(cat_id), None)
    self.save()

def add_estudiante(self, nombre, cedula, asig_id, grupo_id):
    if not _nombre_valido(nombre): return False, "Nombre inválido. Solo letras, espacios y
    guiones. Sin '@' ni símbolos."
    if not cedula: return False, "La cédula no puede estar vacía."
    if not _cedula_valida(cedula): return False, "Cédula inválida. Formato panameño: 8-888-8888 ·
    E-8-12345 · N-20-12345"
    if not self.get_asignatura(asig_id): return False, "Asignatura inválida."
    if not self.get_grupo(grupo_id): return False, "Grupo inválido."
    if any(e.get("cedula") == cedula for e in self.estudiantes):
        return False, "Ya existe un estudiante con esa cédula."
    self.estudiantes.append({"id": self._new_id(self.estudiantes), "nombre": nombre,
                             "cedula": cedula, "asig_id": asig_id, "grupo_id": grupo_id})
    self.save(); return True, "Estudiante agregado."

def update_estudiante(self, est_id, nombre, cedula, grupo_id):
    est = self.get_estudiante(est_id)
    if not est: return False, "No encontrado."
    if not _nombre_valido(nombre): return False, "Nombre inválido. Solo letras, espacios y
    guiones. Sin '@' ni símbolos."
    if not cedula: return False, "La cédula no puede estar vacía."
    if not _cedula_valida(cedula): return False, "Cédula inválida. Formato panameño: 8-888-8888 ·
    E-8-12345 · N-20-12345"
    if not self.get_grupo(grupo_id): return False, "Grupo inválido."
    if any(e.get("cedula") == cedula and e["id"] != est_id for e in self.estudiantes):
        return False, "Ya existe otro estudiante con esa cédula."
    est["nombre"] = nombre; est["cedula"] = cedula; est["grupo_id"] = grupo_id
    self.save(); return True, "Estudiante actualizado."

def delete_estudiante(self, est_id):
    self.estudiantes = [e for e in self.estudiantes if e["id"] != est_id]
    self.notas.pop(str(est_id), None)
    self.save()

def add_nota(self, est_id, cat_id, nota, nombre_nota=""):
    v, err = self._parse_nota(nota)
    if err: return False, err
    est = self.get_estudiante(est_id); cat = self.get_categoria(cat_id)
    if not est or not cat: return False, "Estudiante o categoría no encontrados."
    if est["asig_id"] != cat["asig_id"]: return False, "Categoría no pertenece a la asignatura."
    idx = len(self.notas.get(str(est_id), {})) + 1
    self.notas.setdefault(str(est_id), {}).setdefault(str(cat_id), []).append(
        {"nombre": nombre_nota.strip() or f"Nota {idx}", "valor": round(v, 2)})
    self.save(); return True, "Nota agregada."

def remove_nota(self, est_id, cat_id, idx):
    try:
        self.notas[str(est_id)][str(cat_id)].pop(idx)

```

```

        self.save(); return True
    except Exception: return False

def update_nota(self, est_id, cat_id, idx, nueva_nota, nuevo_nombre=""):
    v, err = self._parse_nota(nueva_nota)
    if err: return False, err
    try:
        cur = self.notas[str(est_id)][str(cat_id)][idx]
        old = cur.get("nombre", f"Nota {idx+1}") if isinstance(cur, dict) else f"Nota {idx+1}"
        self.notas[str(est_id)][str(cat_id)][idx] = {
            "nombre": nuevo_nombre.strip() or old, "valor": round(v, 2)}
        self.save(); return True, "Nota actualizada."
    except Exception: return False, "No se pudo actualizar."

def _vcmd(widget, max_len, chars=None):
    def ok(s):
        if len(s) > max_len: return False
        if chars and s: return all(c in chars for c in s)
        return True
    return (widget.register(ok), "%P")

def _vcmd_num(widget, max_len=5):
    def ok(s): return s == "" or (len(s) <= max_len and s.replace(".", "", 1).isdigit())
    return (widget.register(ok), "%P")

def _vcmd_nombre(widget, max_len):
    def ok(s):
        if len(s) > max_len: return False
        if "@" in s: return False
        return all(c in NOMBRE_CHARS for c in s)
    return (widget.register(ok), "%P")

def _vcmd_cedula(widget, max_len):
    def ok(s):
        if len(s) > max_len: return False
        return all(c in CEDULA_CHARS for c in s)
    return (widget.register(ok), "%P")

def _btn_limpiar(parent, fields):
    def clear():
        for f in fields:
            if isinstance(f, ctk.CTkEntry): f.delete(0, "end")
    ctk.CTkButton(parent, text="Limpiar campos", fg_color=BG_PANEL, text_color=TEXT_DIM,
        border_width=1, border_color=BORDER, height=36,
        command=clear).pack(fill="x", padx=14, pady=(0, 10))

def _crud_btns(parent, on_edit, on_del, h=28, py=6):
    ctk.CTkButton(parent, text="X", width=30, height=h, fg_color=DANGER,
        command=on_del).pack(side="right", padx=4, pady=py)
    ctk.CTkButton(parent, text="↩", width=30, height=h, fg_color=ACCENT,
        command=on_edit).pack(side="right", pady=py)

def _page_header(parent, title):
    top = ctk.CTkFrame(parent, fg_color=BG_PANEL, height=60, corner_radius=0)
    top.pack(fill="x"); top.pack_propagate(False)

```

```

    ctk.CTkLabel(top, text=title, font=("Segoe UI", 20, "bold"),
                 text_color=TEXT_WHITE).pack(side="left", padx=20, pady=16)

def _two_col(parent):
    main = ctk.CTkFrame(parent, fg_color="transparent")
    main.pack(fill="both", expand=True, padx=18, pady=18)
    main.columnconfigure(0, weight=1); main.columnconfigure(1, weight=1)
    def _card(col, padx):
        f = ctk.CTkFrame(main, fg_color=BG_CARD, border_width=1,
                         border_color=BORDER, corner_radius=12)
        f.grid(row=0, column=col, padx=padx, sticky="nsew"); return f
    return _card(0, (0, 8)), _card(1, (8, 0))

def _ok_or_err(result, on_ok=None):
    ok, msg = result
    if not ok: messagebox.showerror("Error", msg)
    elif on_ok: on_ok()
    return ok

def _sync_opt(om, vals):
    om.configure(values=vals)
    if om.get() not in vals: om.set(vals[0])

def _asig_opts(data):
    return [f'{{a["id"]}}{a["nombre"]}' for a in data.asignaturas] or ["—"]

def _asig_id_from(widget):
    raw = widget.get()
    return int(raw.split("|")[0]) if raw != "—" and "|" in raw else None

def _sec_title(parent, text):
    ctk.CTkLabel(parent, text=text, font=("Segoe UI", 14, "bold"),
                 text_color=TEXT_WHITE).pack(anchor="w", padx=14, pady=(14, 8))

def _clear_list(widget, empty_msg=None):
    for w in widget.winfo_children(): w.destroy()
    if empty_msg:
        ctk.CTkLabel(widget, text=empty_msg, text_color=TEXT_DIM).pack(anchor="w", padx=10,
        pady=10)

def _nota_estado(parent, nota):
    ctk.CTkLabel(parent, text="Aprobado" if nota >= 71 else "Reprobado",
                 text_color=SUCCESS if nota >= 71 else DANGER).pack(side="left", padx=8)

class _BaseDialog(ctk.CTkToplevel):
    def __init__(self, parent, title):
        super().__init__(parent)
        self.title(title); self.configure(fg_color=BG_DARK)
        self.resizable(False, False); self.transient(parent); self.grab_set()
        ctk.CTkLabel(self, text=title, font=("Segoe UI", 16, "bold"),
                     text_color=TEXT_WHITE).pack(pady=(20, 12), padx=24)

    def _label(self, text):
        ctk.CTkLabel(self, text=text, font=("Segoe UI", 11),
                     text_color=TEXT_DIM).pack(anchor="w", padx=24)

```



```

def _entry(self, valor="", max_len=60, chars=None, num=False, nombre=False, cedula=False,
width=320):
    if num:
        kw = {"validate": "key", "validatecommand": _vcmd_num(self, max_len)}
    elif nombre:
        kw = {"validate": "key", "validatecommand": _vcmd_nombre(self, max_len)}
    elif cedula:
        kw = {"validate": "key", "validatecommand": _vcmd_cedula(self, max_len)}
    else:
        kw = {"validate": "key", "validatecommand": _vcmd(self, max_len, chars)}
    e = ctk.CTkEntry(self, width=width, height=38, **kw)
    e.insert(0, str(valor)); e.pack(padx=24, pady=(2, 10))
    return e

```

```

def _botones(self, on_save):
    row = ctk.CTkFrame(self, fg_color="transparent"); row.pack(pady=(6, 20), padx=24)
    ctk.CTkButton(row, text="Guardar", fg_color=SUCCESS, width=140,
command=on_save).pack(side="left", padx=6)
    ctk.CTkButton(row, text="Cancelar", fg_color=DANGER, width=140,
command=self.destroy).pack(side="left", padx=6)

```

```

class EditAsigDialog(_BaseDialog):
    def __init__(self, parent, asig, on_save):
        super().__init__(parent, "Editar Asignatura")
        self._label("Nombre de la asignatura:")
        self.nombre = self._entry(asig["nombre"], LIM["nombre_asig"])
        self._label(f"Código único (ej. MAT101, máx. {LIM['codigo_asig']} car.):")
        self.codigo = self._entry(asig["codigo"], LIM["codigo_asig"], ALNUM_GUION)
        self._botones(lambda: (on_save(self.nombre.get().strip(), self.codigo.get().strip()),
self.destroy()))

```

```

class EditGrupoDialog(_BaseDialog):
    def __init__(self, parent, grupo, on_save):
        super().__init__(parent, "Editar Grupo")
        self._label("Nombre del grupo (ej. Grupo A, Lunes 7am):")
        self.nombre = self._entry(grupo["nombre"], LIM["nombre_grupo"])
        self._label("Modalidad de cursado:")
        self.modal = ctk.CTkOptionMenu(self, values=["presencial", "distancia"], width=320)
        self.modal.set(grupo["modalidad"]); self.modal.pack(padx=24, pady=(2, 14))
        self._botones(lambda: (on_save(self.nombre.get().strip(), self.modal.get()), self.destroy()))

```

```

class EditCategoriaDialog(_BaseDialog):
    def __init__(self, parent, cat, on_save):
        super().__init__(parent, "Editar Categoría")
        self._label("Nombre de la categoría (ej. Tareas Semana 1):")
        self.nombre = self._entry(cat["nombre"], LIM["nombre_cat"])
        self._label("Porcentaje que aporta a la nota final (1-100):")
        self.pct = self._entry(str(cat["porcentaje"]), LIM["pct"], num=True)
        self._label("Tipo de evaluación:")
        self.tipo = ctk.CTkOptionMenu(self, values=TIPOS_EVALUACION, width=320)
        self.tipo.set(cat.get("tipo", "Tareas")); self.tipo.pack(padx=24, pady=(2, 6))
        ctk.CTkLabel(self, text="Semestral: mínimo 30% · un tipo por categoría.",

```

```

        font=("Segoe UI", 10), text_color=WARNING).pack(anchor="w", padx=24, pady=(0,
10))
        self._botones(lambda: (on_save(self.nombre.get().strip(),
                                self.pct.get().strip(), self.tipo.get()), self.destroy()))

```

```

class EditEstudianteDialog(_BaseDialog):
    def __init__(self, parent, est, grupos, on_save):
        super().__init__(parent, "Editar Estudiante")
        self._label("Nombre completo del estudiante (sin '@' ni símbolos):")
        self.nombre = self._entry(est["nombre"], LIM["nombre_est"], nombre=True)
        self._label("Cédula panameña (ej. 8-888-8888 · E-8-12345 · N-20-12345):")
        self.cedula = self._entry(est.get("cedula", ""), LIM["cedula"], cedula=True)
        self._label("Grupo al que pertenece:")
        opts = [f"{g['id']}|{g['nombre']}" for g in grupos] or ["—"]
        self.grupo = ctk.CTkOptionMenu(self, values=opts, width=320)
        self.grupo.set(next((f"{g['id']}|{g['nombre']}" for g in grupos
                             if g["id"] == est["grupo_id"]), opts[0]))
        self.grupo.pack(padx=24, pady=(2, 14))
        self._botones(lambda: self._guardar(on_save))

    def _guardar(self, on_save):
        raw = self.grupo.get()
        gid = int(raw.split("|")[0]) if "|" in raw else None
        on_save(self.nombre.get().strip(), self.cedula.get().strip(), gid)
        self.destroy()

```

```

class EditNotaDialog(_BaseDialog):
    def __init__(self, parent, nota_dict, on_save):
        super().__init__(parent, "Editar Nota")
        nd = nota_dict if isinstance(nota_dict, dict) else {}
        self._label("Etiqueta de la nota (ej. Tarea 1, Laboratorio 3):")
        self.nombre = self._entry(nd.get("nombre", ""), LIM["nombre_nota"], width=280)
        self._label("Valor numérico de la nota (0 – 100):")
        self.valor = self._entry(str(nd.get("valor", 0) if nd else float(nota_dict)),
                                LIM["valor_nota"], num=True, width=280)
        self._botones(lambda: (on_save(self.valor.get().strip(),
                                         self.nombre.get().strip()), self.destroy()))

```

```

class LoginFrame(ctk.CTkFrame):
    def __init__(self, parent, on_success):
        super().__init__(parent, fg_color=BG_DARK, corner_radius=0)
        self.on_success = on_success; self._logo_img = None
        self._build()

    def _build(self):
        self.place(relx=0, rely=0, relwidth=1, relheight=1)
        ctk.CTkFrame(self, fg_color="#0a0a18", corner_radius=0).place(relx=0, rely=0, relwidth=1,
relheight=1)

        card = ctk.CTkFrame(self, fg_color=BG_CARD, corner_radius=24,
                             border_width=1, border_color="#3a3a5e", width=420, height=530)
        card.place(relx=0.5, rely=0.5, anchor="center")
        card.pack_propagate(False)

```

```

logo_ok = False
if PIL_OK and os.path.exists(LOGO_PATH):
    try:
        img = Image.open(LOGO_PATH).convert("RGBA").resize((100, 100), Image.LANCZOS)
        self._logo_img = ctk.CTkImage(light_image=img, dark_image=img, size=(100, 100))
        ctk.CTkLabel(card, image=self._logo_img, text="").pack(pady=(30, 6))
        logo_ok = True
    except Exception: pass
if not logo_ok:
    ctk.CTkLabel(card, text="UA", font=("Segoe UI", 44, "bold"),
                 text_color=ACCENT).pack(pady=(30, 6))

    ctk.CTkLabel(card, text="Sistema Académico",
                 font=("Segoe UI", 22, "bold"), text_color=TEXT_WHITE).pack(pady=(4, 2))
    ctk.CTkLabel(card, text="Ingresa tus credenciales para continuar",
                 font=("Segoe UI", 11), text_color=TEXT_DIM).pack(pady=(0, 22))

    ctk.CTkLabel(card, text="Usuario", font=("Segoe UI", 11),
                 text_color=TEXT_DIM).pack(anchor="w", padx=40)
    self.user = ctk.CTkEntry(card, placeholder_text="Escribe tu nombre de usuario",
                             width=340, height=42, corner_radius=10)
    self.user.pack(pady=(2, 10))

    ctk.CTkLabel(card, text="Contraseña", font=("Segoe UI", 11),
                 text_color=TEXT_DIM).pack(anchor="w", padx=40)
    self.pwd = ctk.CTkEntry(card, placeholder_text="Escribe tu contraseña", show="*",
                             width=340, height=42, corner_radius=10)
    self.pwd.pack(pady=(2, 6))

    self.err = ctk.CTkLabel(card, text="", font=("Segoe UI", 11), text_color=DANGER)
    self.err.pack(pady=(4, 0))

    ctk.CTkButton(card, text="Ingresar", width=340, height=44, fg_color=ACCENT,
                  hover_color="#3a5abf", font=("Segoe UI", 13, "bold"), corner_radius=10,
                  command=self.login).pack(pady=16)
    ctk.CTkLabel(card, text="Demo: admin / 1234",
                 font=("Segoe UI", 10), text_color=TEXT_DIM).pack(pady=(0, 20))

    self.pwd.bind("<Return>", lambda e: self.login())
    self.user.bind("<Return>", lambda e: self.pwd.focus())

def login(self):
    u = self.user.get().strip()
    if u == "admin" and self.pwd.get().strip() == "1234":
        self.destroy(); self.on_success(u)
    else:
        self.err.configure(text="Usuario o contraseña incorrectos.")
        self.pwd.delete(0, "end")

class Sidebar(ctk.CTkFrame):
    def __init__(self, parent, on_nav, on_logout, username="Usuario"):
        super().__init__(parent, fg_color=SIDEBAR_BG, corner_radius=0, width=220)
        self.on_nav = on_nav
        self.on_logout = on_logout

```

```

self.username = username
self.buttons = {}
self.active = None
self._logo_img = None
self._avatar_img = None
self._load_images()
self._build()

def _load_images(self):
    if not PIL_OK: return
    if os.path.exists(LOGO_PATH):
        try:
            img = Image.open(LOGO_PATH).convert("RGBA").resize((48, 48), Image.LANCZOS)
            self._logo_img = ctk.CTkImage(light_image=img, dark_image=img, size=(48, 48))
        except Exception: pass
    if os.path.exists(USUARIO_PATH):
        try:
            img = Image.open(USUARIO_PATH).convert("RGBA").resize((36, 36), Image.LANCZOS)
            self._avatar_img = ctk.CTkImage(light_image=img, dark_image=img, size=(36, 36))
        except Exception: pass

def _build(self):
    self.pack_propagate(False)

    header = ctk.CTkFrame(self, fg_color=BG_CARD, corner_radius=0)
    header.pack(fill="x")
    if self._logo_img:
        ctk.CTkLabel(header, image=self._logo_img, text="").pack(pady=(18, 6))
    else:
        ctk.CTkLabel(header, text="UA", font=("Segoe UI", 26, "bold"),
                      text_color=ACCENT).pack(pady=(18, 6))
    ctk.CTkLabel(header, text="Sistema Académico",
                 font=("Segoe UI", 13, "bold"), text_color=TEXT_WHITE).pack(pady=(0, 16))

    ctk.CTkFrame(self, height=1, fg_color=BORDER).pack(fill="x")

    nav = ctk.CTkFrame(self, fg_color="transparent")
    nav.pack(fill="x", pady=(8, 0))
    for name in ["Dashboard", "Asignaturas", "Categorías", "Estudiantes", "Notas", "Cifrado"]:
        btn = ctk.CTkButton(nav, text=name, anchor="w", height=42,
                           fg_color="transparent", hover_color=BG_CARD,
                           text_color=TEXT_DIM, command=lambda n=name: self.on_nav(n))
        btn.pack(fill="x", padx=12, pady=3)
        self.buttons[name] = btn

    ctk.CTkFrame(self, fg_color="transparent").pack(fill="both", expand=True)

    ctk.CTkFrame(self, height=1, fg_color=BORDER).pack(fill="x", padx=14)
    user_card = ctk.CTkFrame(self, fg_color=BG_CARD, corner_radius=10)
    user_card.pack(fill="x", padx=12, pady=10)
    row = ctk.CTkFrame(user_card, fg_color="transparent")
    row.pack(fill="x", padx=10, pady=10)
    if self._avatar_img:
        ctk.CTkLabel(row, image=self._avatar_img, text="",
                     fg_color=BG_PANEL, corner_radius=18).pack(side="left", padx=(0, 8))
    else:

```

```

        ctk.CTkLabel(row, text="UA", font=("Segoe UI", 14, "bold"),
                    text_color=ACCENT, width=36, height=36,
                    fg_color=BG_PANEL, corner_radius=18).pack(side="left", padx=(0, 8))
    info = ctk.CTkFrame(row, fg_color="transparent")
    info.pack(side="left", fill="x", expand=True)
    ctk.CTkLabel(info, text=self.username.capitalize(),
                font=("Segoe UI", 12, "bold"), text_color=TEXT_WHITE,
                anchor="w").pack(anchor="w")
    ctk.CTkLabel(info, text="Administrador",
                font=("Segoe UI", 9), text_color=TEXT_DIM,
                anchor="w").pack(anchor="w")

    ctk.CTkButton(self, text="Cerrar sesión", height=38, fg_color="transparent",
                  hover_color=DANGER, text_color=TEXT_DIM,
                  command=self.on_logout).pack(fill="x", padx=12, pady=(0, 12))

def set_active(self, name):
    if self.active and self.active in self.buttons:
        self.buttons[self.active].configure(fg_color="transparent", text_color=TEXT_DIM)
    self.active = name
    if name in self.buttons:
        self.buttons[name].configure(fg_color=ACCENT, text_color="ffffff")

class DashboardPage(ctk.CTkFrame):
    def __init__(self, parent, app):
        super().__init__(parent, fg_color=BG_DARK)
        self.app = app; self.stats = {}; self._build()

    def _build(self):
        _page_header(self, "Dashboard")

        stats_row = ctk.CTkFrame(self, fg_color="transparent")
        stats_row.pack(fill="x", padx=18, pady=18)
        for i, (key, lbl) in enumerate([("asignaturas", "Asignaturas"), ("grupos", "Grupos"),
                                       ("categorias", "Categorías"), ("estudiantes", "Estudiantes")]):
            card = ctk.CTkFrame(stats_row, fg_color=BG_CARD, border_width=1,
                               border_color=BORDER, corner_radius=12)
            card.grid(row=0, column=i, padx=6, sticky="nsew")
            stats_row.columnconfigure(i, weight=1)
            v = ctk.CTkLabel(card, text="0", font=("Segoe UI", 28, "bold"), text_color=ACCENT)
            v.pack(pady=(18, 4))
            ctk.CTkLabel(card, text=lbl, font=("Segoe UI", 11), text_color=TEXT_DIM).pack(pady=(0,
16))

            self.stats[key] = v

        bottom = ctk.CTkFrame(self, fg_color="transparent")
        bottom.pack(fill="both", expand=True, padx=18, pady=(0, 18))
        bottom.columnconfigure(0, weight=1); bottom.columnconfigure(1, weight=1)

        left = ctk.CTkFrame(bottom, fg_color=BG_CARD, border_width=1, border_color=BORDER,
corner_radius=12)
        left.grid(row=0, column=0, padx=(0, 8), sticky="nsew")
        right = ctk.CTkFrame(bottom, fg_color=BG_CARD, border_width=1, border_color=BORDER,
corner_radius=12)
        right.grid(row=0, column=1, padx=(8, 0), sticky="nsew")

```

```

        _sec_title(left, "Estado por asignatura")
        self.pct_scroll = ctk.CTkScrollableFrame(left, fg_color="transparent", corner_radius=0)
        self.pct_scroll.pack(fill="both", expand=True, padx=14, pady=(0, 14))

        _sec_title(right, "Estudiantes recientes")
        self.recent = ctk.CTkTextbox(right, fg_color=BG_PANEL, text_color=TEXT_WHITE,
                                     border_width=0, font=("Consolas", 12))
        self.recent.pack(fill="both", expand=True, padx=14, pady=(0, 14))

    def refresh(self):
        d = self.app.data
        for key in ("asignaturas", "grupos", "categorias", "estudiantes"):
            self.stats[key].configure(text=str(len(getattr(d, key))))

        _clear_list(self.pct_scroll, "Sin asignaturas aún." if not d.asignaturas else None)
        if d.asignaturas:
            for a in d.asignaturas:
                pct = d.total_pct(a["id"])
                cats = d.categorias_de_asignatura(a["id"])
                tiene_sem = any(c["tipo"] == "Semestral" for c in cats)
                color = SUCCESS if pct == 100 and tiene_sem else (WARNING if pct > 0 else
TEXT_DIM)
                row = ctk.CTkFrame(self.pct_scroll, fg_color="transparent"); row.pack(fill="x", pady=4)
                ctk.CTkLabel(row, text=a["codigo"], font=("Segoe UI", 11, "bold"),
                             text_color=TEXT_WHITE, width=80, anchor="w").pack(side="left")
                bar = ctk.CTkProgressBar(row, height=10, progress_color=color)
                bar.pack(side="left", fill="x", expand=True, padx=6)
                bar.set(min(pct, 100) / 100)
                ctk.CTkLabel(row, text=f"{pct}%", font=("Segoe UI", 10),
                             text_color=color, width=42).pack(side="left")
                ctk.CTkLabel(row, text="Sem OK" if tiene_sem else "Sin Sem",
                             font=("Segoe UI", 9),
                             text_color=SUCCESS if tiene_sem else DANGER, width=60).pack(side="left")

        self.recent.configure(state="normal")
        self.recent.delete("1.0", "end")
        for e in d.estudiantes[:8]:
            asig = d.get_asignatura(e["asig_id"])
            nota = d.nota_final_est(e["id"])
            self.recent.insert("end",
                              f"- {e['nombre']} [{e.get('cedula', '')}] "
                              f"[{asig['codigo']} if asig else '?'] | "
                              f"Final: {nota if nota is not None else '—'}\n")
        if not d.estudiantes:
            self.recent.insert("end", "Sin estudiantes registrados.\n")
        self.recent.configure(state="disabled")

class AsignaturasPage(ctk.CTkFrame):
    def __init__(self, parent, app):
        super().__init__(parent, fg_color=BG_DARK)
        self.app = app; self._build()

    def _build(self):
        _page_header(self, "Asignaturas y Grupos")

```

```

left, right = _two_col(self)

_sec_title(left, "Nueva asignatura")
self.asig_nombre = ctk.CTkEntry(
    left,
    placeholder_text=f"Nombre completo de la asignatura (máx. {LIM['nombre_asig']} car.)",
    height=40, validate="key",
    validatecommand=_vcmd(self, LIM["nombre_asig"]))
self.asig_nombre.pack(fill="x", padx=14, pady=6)
self.asig_codigo = ctk.CTkEntry(
    left,
    placeholder_text=f"Código único, ej. MAT101 (máx. {LIM['codigo_asig']} car., alfanumérico)",
    height=40, validate="key",
    validatecommand=_vcmd(self, LIM["codigo_asig"], ALNUM_GUION))
self.asig_codigo.pack(fill="x", padx=14, pady=6)
ctk.CTkButton(left, text="Agregar asignatura", fg_color=SUCCESS, height=40,
    command=self.add_asignatura).pack(fill="x", padx=14, pady=(6, 4))
_btn_limpiar(left, [self.asig_nombre, self.asig_codigo])
ctk.CTkFrame(left, height=1, fg_color=BORDER).pack(fill="x", padx=14, pady=8)

_sec_title(left, "Nuevo grupo")
self.group_asig = ctk.CTkOptionMenu(left, values=["—"], dynamic_resizing=False)
self.group_asig.pack(fill="x", padx=14, pady=6)
self.group_nombre = ctk.CTkEntry(
    left,
    placeholder_text=f"Nombre del grupo, ej. Grupo A / Lunes 7am (máx. {LIM['nombre_grupo']}
car.)",
    height=40, validate="key",
    validatecommand=_vcmd(self, LIM["nombre_grupo"]))
self.group_nombre.pack(fill="x", padx=14, pady=6)
self.group_modal = ctk.CTkOptionMenu(left, values=["presencial", "distancia"],
dynamic_resizing=False)
self.group_modal.pack(fill="x", padx=14, pady=6)
ctk.CTkButton(left, text="Agregar grupo", fg_color=SUCCESS, height=40,
    command=self.add_grupo).pack(fill="x", padx=14, pady=(6, 4))
_btn_limpiar(left, [self.group_nombre])

_sec_title(right, "Asignaturas registradas")
self.list_box = ctk.CTkScrollableFrame(right, fg_color=BG_PANEL, corner_radius=10)
self.list_box.pack(fill="both", expand=True, padx=14, pady=(0, 14))

def _refresh_group_opts(self):
    opts = _asig_opts(self.app.data)
    _sync_opt(self.group_asig, opts)

def add_asignatura(self):
    if _ok_or_err(self.app.data.add_asignatura(
        self.asig_nombre.get().strip(), self.asig_codigo.get().strip())):
        self.asig_nombre.delete(0, "end"); self.asig_codigo.delete(0, "end")
        self.refresh()

def add_grupo(self):
    raw = self.group_asig.get()
    if raw == "—" or "|" not in raw:
        messagebox.showerror("Error", "Primero crea una asignatura."); return
    if _ok_or_err(self.app.data.add_grupo(int(raw.split("|")[0]),

```

```

        self.group_nombre.get().strip(), self.group_modal.get())):
    self.group_nombre.delete(0, "end"); self.refresh()

def edit_asig(self, asig_id):
    a = self.app.data.get_asignatura(asig_id)
    if not a: return
    EditAsigDialog(self, a, lambda n, c: _ok_or_err(
        self.app.data.update_asignatura(asig_id, n, c), self.refresh))

def edit_grupo(self, grupo_id):
    g = self.app.data.get_grupo(grupo_id)
    if not g: return
    EditGrupoDialog(self, g, lambda n, m: _ok_or_err(
        self.app.data.update_grupo(grupo_id, n, m), self.refresh))

def delete_asig(self, asig_id):
    if messagebox.askyesno("Confirmar", "¿Eliminar asignatura y todos sus datos?"):
        self.app.data.delete_asignatura(asig_id); self.refresh()

def delete_grupo(self, grupo_id):
    if messagebox.askyesno("Confirmar", "¿Eliminar grupo y sus estudiantes?"):
        self.app.data.delete_grupo(grupo_id); self.refresh()

def refresh(self):
    self._refresh_group_opts()
    d = self.app.data
    _clear_list(self.list_box, "Sin asignaturas aún." if not d.asignaturas else None)
    if not d.asignaturas: return
    for a in d.asignaturas:
        card = ctk.CTkFrame(self.list_box, fg_color=BG_PANEL,
                             border_width=1, border_color=BORDER, corner_radius=10)
        card.pack(fill="x", pady=6, padx=2)
        head = ctk.CTkFrame(card, fg_color="transparent")
        head.pack(fill="x", padx=10, pady=(10, 4))
        ctk.CTkLabel(head, text=f"{a['codigo']} | {a['nombre']}",
                     font=("Segoe UI", 13, "bold"), text_color=TEXT_WHITE).pack(side="left")
        _crud_btns(head, lambda aid=a["id"]: self.edit_asig(aid),
                   lambda aid=a["id"]: self.delete_asig(aid))
        groups = d.grupos_de_asignatura(a["id"])
        if not groups:
            ctk.CTkLabel(card, text="Sin grupos.", text_color=TEXT_DIM).pack(anchor="w",
                padx=10, pady=(0, 10))
        else:
            for g in groups:
                row = ctk.CTkFrame(card, fg_color=BG_CARD, corner_radius=8)
                row.pack(fill="x", padx=10, pady=4)
                ctk.CTkLabel(row, text=f"{g['nombre']} ({g['modalidad']})",
                             text_color=TEXT_WHITE).pack(side="left", padx=10, pady=8)
                _crud_btns(row, lambda gid=g["id"]: self.edit_grupo(gid),
                           lambda gid=g["id"]: self.delete_grupo(gid), py=4)

class CategoriasPage(ctk.CTkFrame):
    def __init__(self, parent, app):
        super().__init__(parent, fg_color=BG_DARK)
        self.app = app; self._build()

```



```

def _build(self):
    _page_header(self, "Categorías de Evaluación")
    left, right = _two_col(self)

    _sec_title(left, "Nueva categoría")
    ctk.CTkLabel(left, text="Asignatura a la que pertenece esta categoría:",
                  font=("Segoe UI", 11), text_color=TEXT_DIM).pack(anchor="w", padx=14)
    self.cat_asig = ctk.CTkOptionMenu(left, values=["—"], dynamic_resizing=False,
                                      command=lambda _: self._refresh_pct_bar())
    self.cat_asig.pack(fill="x", padx=14, pady=(2, 8))
    self.cat_nombre = ctk.CTkEntry(
        left,
        placeholder_text=f"Nombre de la categoría, ej. Tareas parcial 1 (máx. {LIM['nombre_cat']}
car.)",
        height=40, validate="key",
        validatecommand=_vcmd(self, LIM["nombre_cat"]))
    self.cat_nombre.pack(fill="x", padx=14, pady=6)
    self.cat_pct = ctk.CTkEntry(
        left,
        placeholder_text="Porcentaje que aporta a la nota final, ej. 20",
        height=40, validate="key",
        validatecommand=_vcmd_num(self, LIM["pct"]))
    self.cat_pct.pack(fill="x", padx=14, pady=6)
    ctk.CTkLabel(left, text="Tipo de evaluación:",
                  font=("Segoe UI", 11), text_color=TEXT_DIM).pack(anchor="w", padx=14)
    self.cat_tipo = ctk.CTkOptionMenu(left, values=TIPOS_EVALUACION,
dynamic_resizing=False)
    self.cat_tipo.pack(fill="x", padx=14, pady=(2, 4))
    self.cat_tipo.set("Tareas")
    ctk.CTkLabel(left, text="Semestral: mínimo 30% · obligatorio · un tipo por categoría.",
                  font=("Segoe UI", 10), text_color=WARNING,
                  wraplength=280, justify="left").pack(anchor="w", padx=14, pady=(0, 6))
    ctk.CTkButton(left, text="Agregar categoría", fg_color=SUCCESS, height=40,
                  command=self.add_categoria).pack(fill="x", padx=14, pady=(4, 4))
    _btn_limpiar(left, [self.cat_nombre, self.cat_pct])
    self.pct_label = ctk.CTkLabel(left, text="", text_color=TEXT_DIM)
    self.pct_label.pack(anchor="w", padx=14, pady=(6, 2))
    self.pct_bar = ctk.CTkProgressBar(left, height=12)
    self.pct_bar.pack(fill="x", padx=14, pady=(0, 14))
    self.pct_bar.set(0)

    top_r = ctk.CTkFrame(right, fg_color="transparent")
    top_r.pack(fill="x", padx=14, pady=(14, 4))
    ctk.CTkLabel(top_r, text="Categorías de:", font=("Segoe UI", 14, "bold"),
                  text_color=TEXT_WHITE).pack(side="left")
    self.filter_asig = ctk.CTkOptionMenu(top_r, values=["—"], dynamic_resizing=False,
                                      width=200, command=lambda _: self._refresh_list())
    self.filter_asig.pack(side="left", padx=8)
    self.list_box = ctk.CTkScrollableFrame(right, fg_color=BG_PANEL, corner_radius=10)
    self.list_box.pack(fill="both", expand=True, padx=14, pady=(0, 14))

def _refresh_pct_bar(self):
    aid = _asig_id_from(self.cat_asig)
    if aid is None:
        self.pct_label.configure(text="Selecciona una asignatura"); self.pct_bar.set(0); return

```

```

pct = self.app.data.total_pct(aid)
color = SUCCESS if pct == 100 else (WARNING if pct > 0 else ACCENT)
self.pct_label.configure(text=f"Total asignado: {pct}%")
self.pct_bar.configure(progress_color=color)
self.pct_bar.set(min(pct, 100) / 100)

def add_categoria(self):
    aid = _asig_id_from(self.cat_asig)
    if aid is None: messagebox.showerror("Error", "Selecciona una asignatura."); return
    def on_ok():
        self.cat_nombre.delete(0, "end"); self.cat_pct.delete(0, "end")
        self.cat_tipo.set("Tareas"); self.refresh()
    _ok_or_err(self.app.data.add_categoria(
        aid, self.cat_nombre.get().strip(), self.cat_pct.get().strip(), self.cat_tipo.get()), on_ok)

def edit_categoria(self, cat_id):
    c = self.app.data.get_categoria(cat_id)
    if not c: return
    EditCategoriaDialog(self, c, lambda n, p, t: _ok_or_err(
        self.app.data.update_categoria(cat_id, n, p, t), self.refresh))

def delete_categoria(self, cat_id):
    if messagebox.askyesno("Confirmar", "¿Eliminar categoría y sus notas?"):
        self.app.data.delete_categoria(cat_id); self.refresh()

def _refresh_list(self):
    d = self.app.data
    fid = _asig_id_from(self.filter_asig)
    _clear_list(self.list_box)
    if fid is None:
        ctk.CTkLabel(self.list_box, text="Selecciona una asignatura.",
            text_color=TEXT_DIM).pack(anchor="w", padx=10, pady=10); return
    cats = d.categorias_de_asignatura(fid)
    if not cats:
        ctk.CTkLabel(self.list_box, text="Sin categorías para esta asignatura.",
            text_color=TEXT_DIM).pack(anchor="w", padx=10, pady=10); return
    pct_t = d.total_pct(fid)
    tiene_sem = any(c["tipo"] == "Semestral" for c in cats)
    ct = SUCCESS if pct_t == 100 and tiene_sem else (WARNING if pct_t > 0 else
TEXT_DIM)
    ctk.CTkLabel(self.list_box, text=f"Total: {pct_t}%",
        font=("Segoe UI", 11, "bold"), text_color=ct).pack(anchor="e", padx=10, pady=(4, 2))
    ctk.CTkLabel(self.list_box,
        text="Semestral incluido" if tiene_sem else "Falta Semestral (obligatorio)",
        font=("Segoe UI", 10),
        text_color=SUCCESS if tiene_sem else DANGER).pack(anchor="e", padx=10,
pady=(0, 6))
    for c in cats:
        row = ctk.CTkFrame(self.list_box, fg_color=BG_PANEL, corner_radius=8)
        row.pack(fill="x", pady=4, padx=2)
        tc = WARNING if c.get("tipo") == "Semestral" else TEXT_DIM
        ctk.CTkLabel(row, text=c["nombre"], text_color=TEXT_WHITE).pack(side="left", padx=10,
pady=8)
        ctk.CTkLabel(row, text=f"[{c.get('tipo','?')}]",
            text_color=tc, font=("Segoe UI", 10)).pack(side="left", padx=4)
        ctk.CTkLabel(row, text=f"[{c['porcentaje']}]", text_color=TEXT_DIM).pack(side="left",

```

```

padx=8)
    _crud_btns(row, lambda cid=c["id"]: self.edit_categoria(cid),
               lambda cid=c["id"]: self.delete_categoria(cid))

def refresh(self):
    vals = _asig_opts(self.app.data)
    _sync_opt(self.cat_asig, vals); _sync_opt(self.filter_asig, vals)
    self._refresh_pct_bar(); self._refresh_list()

class EstudiantesPage(ctk.CTkFrame):
    def __init__(self, parent, app):
        super().__init__(parent, fg_color=BG_DARK)
        self.app = app; self._build()

    def _build(self):
        _page_header(self, "Estudiantes")
        left, right = _two_col(self)

        _sec_title(left, "Registrar estudiante")
        self.est_nombre = ctk.CTkEntry(
            left,
            placeholder_text="Nombre completo, ej. Juan Pérez (máx. {LIM['nombre_est']} car., sin
'@')",
            height=40, validate="key",
            validatecommand=_vcmd_nombre(self, LIM["nombre_est"]))
        self.est_nombre.pack(fill="x", padx=14, pady=6)
        self.est_cedula = ctk.CTkEntry(
            left,
            placeholder_text="Cédula panameña, ej. 8-888-8888 · E-8-12345 · N-20-12345",
            height=40, validate="key",
            validatecommand=_vcmd_cedula(self, LIM["cedula"]))
        self.est_cedula.pack(fill="x", padx=14, pady=6)
        self.asig_menu = ctk.CTkOptionMenu(left, values=["—"], dynamic_resizing=False,
                                           command=self.on_asig_change)
        self.asig_menu.pack(fill="x", padx=14, pady=6)
        self.group_menu = ctk.CTkOptionMenu(left, values=["—"], dynamic_resizing=False)
        self.group_menu.pack(fill="x", padx=14, pady=6)
        ctk.CTkButton(left, text="Registrar estudiante", fg_color=SUCCESS, height=40,
                      command=self.add_estudiante).pack(fill="x", padx=14, pady=(6, 4))
        _btn_limpiar(left, [self.est_nombre, self.est_cedula])

        _sec_title(right, "Lista de estudiantes")
        self.list_box = ctk.CTkScrollableFrame(right, fg_color=BG_PANEL, corner_radius=10)
        self.list_box.pack(fill="both", expand=True, padx=14, pady=(0, 14))

    def on_asig_change(self, _=None):
        raw = self.asig_menu.get()
        if raw == "—" or "|" not in raw:
            self.group_menu.configure(values=["—"]); self.group_menu.set("—"); return
        gs = self.app.data.grupos_de_asignatura(int(raw.split("|")[0]))
        vals = [f"{g['id']}|{g['nombre']}" for g in gs] or ["—"]
        self.group_menu.configure(values=vals); self.group_menu.set(vals[0])

    def add_estudiante(self):
        ra, rg = self.asig_menu.get(), self.group_menu.get()

```

```

if ra == "—" or rg == "—" or "|" not in ra or "|" not in rg:
    messagebox.showerror("Error", "Selecciona asignatura y grupo."); return
def on_ok():
    self.est_nombre.delete(0, "end"); self.est_cedula.delete(0, "end")
    self.refresh()
    _ok_or_err(self.app.data.add_estudiante(
        self.est_nombre.get().strip(), self.est_cedula.get().strip(),
        int(ra.split("|")[0]), int(rg.split("|")[0])), on_ok)

def edit_estudiante(self, est_id):
    est = self.app.data.get_estudiante(est_id)
    if not est: return
    grupos = self.app.data.grupos_de_asignatura(est["asig_id"])
    EditEstudianteDialog(self, est, grupos, lambda n, c, g: _ok_or_err(
        self.app.data.update_estudiante(est_id, n, c, g), self.refresh))

def delete_estudiante(self, est_id):
    if messagebox.askyesno("Confirmar", "¿Eliminar estudiante y sus notas?"):
        self.app.data.delete_estudiante(est_id); self.refresh()

def refresh(self):
    d = self.app.data
    _sync_opt(self.asig_menu, _asig_opts(d))
    self.on_asig_change()
    _clear_list(self.list_box, "Sin estudiantes aún." if not d.estudiantes else None)
    if not d.estudiantes: return
    for e in d.estudiantes:
        asig = d.get_asignatura(e["asig_id"])
        grupo = d.get_grupo(e["grupo_id"])
        nota = d.nota_final_est(e["id"])
        row = ctk.CTkFrame(self.list_box, fg_color=BG_PANEL, corner_radius=8)
        row.pack(fill="x", pady=4, padx=2)
        ctk.CTkLabel(row,
            text=f"{e['nombre']} [{e.get('cedula','—')}] | "
            f"{asig['codigo'] if asig else '?'} / {grupo['nombre'] if grupo else '?'}",
            text_color=TEXT_WHITE).pack(side="left", padx=10, pady=8)
        ctk.CTkLabel(row, text=f"{nota if nota is not None else '—'}",
            text_color=WARNING if nota is None else TEXT_WHITE).pack(side="left",
padx=10)
        if nota is not None: _nota_estado(row, nota)
        _crud_btns(row, lambda eid=e["id"]: self.edit_estudiante(eid),
            lambda eid=e["id"]: self.delete_estudiante(eid))

class NotesModal(ctk.CTkToplevel):
    def __init__(self, app, est_id):
        super().__init__(app)
        self.app = app; self.est_id = est_id
        self.title("Notas del estudiante")
        self.geometry("560x680"); self.configure(fg_color=BG_DARK)
        self.transient(app); self.grab_set()
        self._build(); self.refresh()

    def _build(self):
        est = self.app.data.get_estudiante(self.est_id)
        nombre = est["nombre"] if est else ""

```

```

ced = est.get("cedula", "") if est else ""
ctk.CTkLabel(self, text=f"Notas de {nombre}",
             font=("Segoe UI", 18, "bold"), text_color=TEXT_WHITE).pack(pady=(16, 2))
ctk.CTkLabel(self, text=f"Cédula: {ced}",
             font=("Segoe UI", 11), text_color=TEXT_DIM).pack(pady=(0, 8))
self.container = ctk.CTkScrollableFrame(self, fg_color=BG_CARD, corner_radius=12)
self.container.pack(fill="both", expand=True, padx=16, pady=10)
ctk.CTkButton(self, text="Cerrar", fg_color=ACCENT, command=self.destroy).pack(pady=(0,
16))

```

```

def refresh(self):
    d = self.app.data
    est = d.get_estudiante(self.est_id)
    _clear_list(self.container)
    if not est: return
    cats = d.categorias_de_asignatura(est["asig_id"])
    if not cats:
        ctk.CTkLabel(self.container,
                     text="Primero define categorías para esta asignatura.",
                     text_color=TEXT_DIM).pack(anchor="w", padx=10, pady=10); return
    ok, razon = d.puede_calcular_final(self.est_id)
    if not ok:
        ctk.CTkLabel(self.container, text=f"Nota final no calculable: {razon}",
                     text_color=WARNING, wraplength=480,
                     justify="left").pack(anchor="w", padx=10, pady=(10, 6))
    for cat in cats:
        block = ctk.CTkFrame(self.container, fg_color=BG_PANEL, corner_radius=10)
        block.pack(fill="x", padx=8, pady=8)
        head = ctk.CTkFrame(block, fg_color="transparent")
        head.pack(fill="x", padx=10, pady=(10, 6))
        ctk.CTkLabel(head, text=f"{cat['nombre']} ({cat['porcentaje']}%)",
                     text_color=TEXT_WHITE).pack(side="left")
        ctk.CTkLabel(head, text=f"[{cat.get('tipo', '?')}]'",
                     text_color=TEXT_DIM, font=("Segoe UI", 10)).pack(side="left", padx=6)
        if cat.get("tipo") == "Semestral":
            ctk.CTkLabel(head, text="Semestral", text_color=WARNING).pack(side="right")

    notes = d.notas.get(str(self.est_id), {}).get(str(cat["id"]), [])
    rn = ctk.CTkFrame(block, fg_color="transparent")
    rn.pack(fill="x", padx=10, pady=(0, 6))
    if notes:
        for idx, n in enumerate(notes):
            item = ctk.CTkFrame(rn, fg_color=BG_CARD, corner_radius=8)
            item.pack(fill="x", pady=3)
            ctk.CTkLabel(item, text=f"{d._nota_nombre(n, idx)}: {d._nota_valor(n)}",
                         text_color=TEXT_WHITE).pack(side="left", padx=10, pady=6)
            _crud_btns(item, lambda i=idx, cid=cat["id"], nd=n: self.edit_note(cid, i, nd),
                       lambda i=idx, cid=cat["id"]: self.remove_note(cid, i), h=26)
    else:
        ctk.CTkLabel(rn, text="Sin notas aún.", text_color=TEXT_DIM).pack(anchor="w")

    form = ctk.CTkFrame(block, fg_color="transparent")
    form.pack(fill="x", padx=10, pady=(0, 10))
    ne = ctk.CTkEntry(
        form,
        placeholder_text=f"Etiqueta, ej. Tarea 1 / Lab. 3 (máx. {LIM['nombre_nota']} car.)",

```

```

        height=34, validate="key",
        validatecommand=_vcmd(self, LIM["nombre_notas"]))
    ne.pack(side="left", fill="x", expand=True, padx=(0, 6))
    ve = ctk.CTkEntry(
        form,
        placeholder_text="Nota (0-100)",
        height=34, width=110,
        validate="key",
        validatecommand=_vcmd_num(self, LIM["valor_notas"]))
    ve.pack(side="left", padx=(0, 6))
    ctk.CTkButton(form, text="Agregar", fg_color=SUCCESS, width=90,
        command=lambda cid=cat["id"], n=ne, v=ve: self.add_note(cid, n,
v)).pack(side="right")

    def add_note(self, cat_id, nombre_entry, valor_entry):
        def on_ok():
            nombre_entry.delete(0, "end"); valor_entry.delete(0, "end")
            self.refresh(); self.app.refresh_all()
        _ok_or_err(self.app.data.add_nota(self.est_id, cat_id,
            valor_entry.get().strip(), nombre_entry.get().strip()), on_ok)

    def edit_note(self, cat_id, idx, nota_dict):
        def on_ok(): self.refresh(); self.app.refresh_all()
        EditNotaDialog(self, nota_dict, lambda v, n: _ok_or_err(
            self.app.data.update_nota(self.est_id, cat_id, idx, v, n), on_ok))

    def remove_note(self, cat_id, idx):
        self.app.data.remove_nota(self.est_id, cat_id, idx)
        self.refresh(); self.app.refresh_all()

class NotesPage(ctk.CTkFrame):
    def __init__(self, parent, app):
        super().__init__(parent, fg_color=BG_DARK)
        self.app = app; self._build()

    def _build(self):
        _page_header(self, "Registro de Notas")
        self.body = ctk.CTkScrollableFrame(self, fg_color="transparent")
        self.body.pack(fill="both", expand=True, padx=18, pady=18)

    def open_modal(self, est_id):
        est = self.app.data.get_estudiante(est_id)
        if not est: return
        if not self.app.data.categorias_de_asignatura(est["asig_id"]):
            messagebox.showwarning("Aviso", "Primero define categorías para la asignatura."); return
        NotesModal(self.app, est_id)

    def refresh(self):
        d = self.app.data
        _clear_list(self.body, "Sin estudiantes registrados." if not d.estudiantes else None)
        if not d.estudiantes: return
        for e in d.estudiantes:
            row = ctk.CTkFrame(self.body, fg_color=BG_CARD, corner_radius=10)
            row.pack(fill="x", padx=4, pady=6)
            asig = d.get_asignatura(e["asig_id"])

```

```

        nota = d.nota_final_est(e["id"])
        ctk.CTkLabel(row,
                    text=f"{e['nombre']} [{e.get('cedula','')}] [{e.get('codigo') if e.get('codigo') else '?'}]",
                    text_color=TEXT_WHITE, font=("Segoe UI", 13, "bold")).pack(side="left", padx=10,
pady=8)
        ctk.CTkLabel(row, text=f"Final: {nota if nota is not None else '—'}",
                    text_color=TEXT_DIM).pack(side="left", padx=12)
        if nota is not None:
            _nota_estado(row, nota)
        else:
            _, razon = d.puede_calcular_final(e["id"])
            ctk.CTkLabel(row, text=razon, text_color=WARNING,
                        font=("Segoe UI", 10)).pack(side="left", padx=6)
            ctk.CTkButton(row, text="Abrir", fg_color=ACCENT, width=80,
                        command=lambda eid=e["id"]: self.open_modal(eid)).pack(side="right", padx=10,
pady=8)

```

```

class CifradoPage(ctk.CTkFrame):
    def __init__(self, parent, app):
        super().__init__(parent, fg_color=BG_DARK)
        self.app = app
        self._sym_key = self._rsa_priv = self._rsa_pub = None
        self._sym_file = self._rsa_file = None
        self._img_candado = self._img_llave = None
        if PIL_OK:
            for attr, path in ((_img_candado, CANDADO_PATH), (_img_llave, LLAVE_PATH)):
                if os.path.exists(path):
                    try:
                        img = Image.open(path).convert("RGBA").resize((22, 22), Image.LANCZOS)
                        setattr(self, attr, ctk.CTkImage(light_image=img, dark_image=img, size=(22, 22)))
                    except Exception: pass
        self._build()

    def _build(self):
        _page_header(self, "Cifrado de Archivos")
        if not CRYPTO_OK:
            warn = ctk.CTkFrame(self, fg_color=BG_CARD, corner_radius=12,
                                border_width=1, border_color=BORDER)
            warn.pack(fill="x", padx=18, pady=18)
            ctk.CTkLabel(warn, text="Librería requerida no instalada. Ejecuta en terminal:",
                        font=("Segoe UI", 13, "bold"), text_color=WARNING).pack(anchor="w", padx=16,
pady=(16, 4))
            ctk.CTkLabel(warn, text="  pip install cryptography",
                        font=("Consolas", 13), text_color=TEXT_WHITE).pack(anchor="w", padx=16,
pady=(0, 16))
            return
        self.tabs = ctk.CTkTabview(self, fg_color=BG_CARD, corner_radius=12)
        self.tabs.pack(fill="both", expand=True, padx=18, pady=18)
        self.tabs.add("Simetrico (Fernet/AES)")
        self.tabs.add("Asimetrico (RSA-2048)")
        self._build_sim(self.tabs.tab("Simetrico (Fernet/AES)"))
        self._build_asim(self.tabs.tab("Asimetrico (RSA-2048)"))

    def _build_log_panel(self, parent, log_attr):
        ctk.CTkLabel(parent, text="Registro de operaciones", font=("Segoe UI", 13, "bold"),

```

```

        text_color=TEXT_WHITE).pack(anchor="w", padx=14, pady=(14, 4))
    log = ctk.CTkTextbox(parent, fg_color=BG_CARD, text_color=TEXT_WHITE,
font=("Consolas", 11))
    log.pack(fill="both", expand=True, padx=14, pady=(0, 8))
    log.configure(state="disabled")
    ctk.CTkButton(parent, text="Limpiar registro", fg_color=BG_PANEL, text_color=TEXT_DIM,
        border_width=1, border_color=BORDER, height=30,
        command=lambda: self._log_clear(log)).pack(fill="x", padx=14, pady=(0, 14))
    setattr(self, log_attr, log)

def _set_key_display(self, val):
    self.sym_key_box.configure(state="normal")
    self.sym_key_box.delete("1.0", "end")
    self.sym_key_box.insert("1.0", val)
    self.sym_key_box.configure(state="disabled")

def _panel(self, parent, col):
    f = ctk.CTkFrame(parent, fg_color=BG_PANEL, corner_radius=10)
    f.grid(row=0, column=col, padx=(0, 8) if col == 0 else (8, 0), pady=8, sticky="nsew")
    return f

def _build_sim(self, p):
    p.columnconfigure(0, weight=1); p.columnconfigure(1, weight=1)
    L = self._panel(p, 0); R = self._panel(p, 1)

    ctk.CTkLabel(L, text="Clave Fernet (Base64)", font=("Segoe UI", 13, "bold"),
        text_color=TEXT_WHITE).pack(anchor="w", padx=14, pady=(14, 4))
    ctk.CTkLabel(L, text="La misma clave cifra y descifra.",
        font=("Segoe UI", 10), text_color=TEXT_DIM).pack(anchor="w", padx=14, pady=(0,
8))

    self.sym_key_box = ctk.CTkTextbox(L, height=56, fg_color=BG_CARD,
        text_color=TEXT_WHITE, font=("Consolas", 10))
    self.sym_key_box.pack(fill="x", padx=14, pady=(0, 8))
    rb = ctk.CTkFrame(L, fg_color="transparent"); rb.pack(fill="x", padx=14, pady=(0, 6))
    ctk.CTkButton(rb, text="Generar clave", fg_color=ACCENT, height=34,
        command=self._sym_gen).pack(side="left", expand=True, fill="x", padx=(0, 4))
    ctk.CTkButton(rb, text="Guardar .key", fg_color=BG_CARD, text_color=TEXT_DIM,
        border_width=1, border_color=BORDER, height=34,
        command=self._sym_save_key).pack(side="left", expand=True, fill="x", padx=(4, 0))
    ctk.CTkButton(L, text="Cargar clave (.key)", fg_color=BG_CARD, text_color=TEXT_DIM,
        border_width=1, border_color=BORDER, height=34,
        command=self._sym_load_key).pack(fill="x", padx=14, pady=(0, 12))
    ctk.CTkFrame(L, height=1, fg_color=BORDER).pack(fill="x", padx=14, pady=4)
    ctk.CTkLabel(L, text="Archivo a cifrar / descifrar", font=("Segoe UI", 13, "bold"),
        text_color=TEXT_WHITE).pack(anchor="w", padx=14, pady=(10, 4))
    self.sym_file_lbl = ctk.CTkLabel(L, text="Ningún archivo seleccionado",
        font=("Segoe UI", 10),
        text_color=TEXT_DIM, wraplength=280, justify="left")
    self.sym_file_lbl.pack(anchor="w", padx=14, pady=(0, 8))
    ctk.CTkButton(L, text="Seleccionar archivo...", fg_color=BG_CARD, text_color=TEXT_DIM,
        border_width=1, border_color=BORDER, height=34,
        command=self._sym_pick).pack(fill="x", padx=14, pady=(0, 8))
    ab = ctk.CTkFrame(L, fg_color="transparent"); ab.pack(fill="x", padx=14, pady=(0, 14))
    ctk.CTkButton(ab, text="Cifrar", image=self._img_candado, compound="left",
        fg_color=SUCCESS, height=40,
        command=self._sym_enc).pack(side="left", expand=True, fill="x", padx=(0, 4))

```



```

        ctk.CTkButton(ab, text="Descifrar", image=self._img_llave, compound="left",
                      fg_color=WARNING, text_color=BG_DARK, height=40,
                      command=self._sym_dec).pack(side="left", expand=True, fill="x", padx=(4, 0))

    self._build_log_panel(R, "sym_log")

def _build_asim(self, p):
    p.columnconfigure(0, weight=1); p.columnconfigure(1, weight=1)
    L = self._panel(p, 0); R = self._panel(p, 1)

    ctk.CTkLabel(L, text="Par de claves RSA-2048", font=("Segoe UI", 13, "bold"),
                  text_color=TEXT_WHITE).pack(anchor="w", padx=14, pady=(14, 4))
    ctk.CTkLabel(L, text="Pública cifra · Privada descifra.",
                  font=("Segoe UI", 10), text_color=TEXT_DIM).pack(anchor="w", padx=14, pady=(0,
8))

    ctk.CTkButton(L, text="Generar par de claves", fg_color=ACCENT, height=36,
                  command=self._rsa_gen).pack(fill="x", padx=14, pady=(0, 6))
    sg = ctk.CTkFrame(L, fg_color="transparent"); sg.pack(fill="x", padx=14, pady=(0, 6))
    ctk.CTkButton(sg, text="Guardar pública", fg_color=BG_CARD, text_color=TEXT_DIM,
                  border_width=1, border_color=BORDER, height=32,
                  command=self._rsa_save_pub).pack(side="left", expand=True, fill="x", padx=(0, 4))
    ctk.CTkButton(sg, text="Guardar privada", fg_color=BG_CARD, text_color=TEXT_DIM,
                  border_width=1, border_color=BORDER, height=32,
                  command=self._rsa_save_priv).pack(side="left", expand=True, fill="x", padx=(4, 0))
    lg = ctk.CTkFrame(L, fg_color="transparent"); lg.pack(fill="x", padx=14, pady=(0, 8))
    ctk.CTkButton(lg, text="Cargar pública", fg_color=BG_CARD, text_color=TEXT_DIM,
                  border_width=1, border_color=BORDER, height=32,
                  command=self._rsa_load_pub).pack(side="left", expand=True, fill="x", padx=(0, 4))
    ctk.CTkButton(lg, text="Cargar privada", fg_color=BG_CARD, text_color=TEXT_DIM,
                  border_width=1, border_color=BORDER, height=32,
                  command=self._rsa_load_priv).pack(side="left", expand=True, fill="x", padx=(4, 0))
    self.rsa_status = ctk.CTkLabel(L, text="Estado: sin claves cargadas.",
                                   font=("Segoe UI", 10), text_color=TEXT_DIM,
                                   wraplength=280, justify="left")
    self.rsa_status.pack(anchor="w", padx=14, pady=(0, 10))
    ctk.CTkFrame(L, height=1, fg_color=BORDER).pack(fill="x", padx=14, pady=4)
    ctk.CTkLabel(L, text="Archivo a cifrar / descifrar", font=("Segoe UI", 13, "bold"),
                  text_color=TEXT_WHITE).pack(anchor="w", padx=14, pady=(10, 4))
    self.rsa_file_lbl = ctk.CTkLabel(L, text="Ningún archivo seleccionado",
                                     font=("Segoe UI", 10),
                                     text_color=TEXT_DIM, wraplength=280, justify="left")
    self.rsa_file_lbl.pack(anchor="w", padx=14, pady=(0, 8))
    ctk.CTkButton(L, text="Seleccionar archivo...", fg_color=BG_CARD, text_color=TEXT_DIM,
                  border_width=1, border_color=BORDER, height=34,
                  command=self._rsa_pick).pack(fill="x", padx=14, pady=(0, 8))
    ab = ctk.CTkFrame(L, fg_color="transparent"); ab.pack(fill="x", padx=14, pady=(0, 14))
    ctk.CTkButton(ab, text="Cifrar (pub)", image=self._img_candado, compound="left",
                  fg_color=SUCCESS, height=40,
                  command=self._rsa_enc).pack(side="left", expand=True, fill="x", padx=(0, 4))
    ctk.CTkButton(ab, text="Descifrar (priv)", image=self._img_llave, compound="left",
                  fg_color=WARNING, text_color=BG_DARK, height=40,
                  command=self._rsa_dec).pack(side="left", expand=True, fill="x", padx=(4, 0))

    self._build_log_panel(R, "rsa_log")

def _log(self, box, msg):

```

```

box.configure(state="normal"); box.insert("end", msg + "\n")
box.configure(state="disabled"); box.see("end")

def _log_clear(self, box):
    box.configure(state="normal"); box.delete("1.0", "end"); box.configure(state="disabled")

def _sym_gen(self):
    try:
        self._sym_key = Fernet.generate_key()
        self._set_key_display(self._sym_key.decode())
        self._log(self.sym_log, "Clave Fernet generada en memoria.")
    except Exception as e: self._log(self.sym_log, f"Error: {e}")

def _sym_save_key(self):
    if not self._sym_key: messagebox.showwarning("Aviso", "Genera o carga una clave primero.");
return
    p = filedialog.asksaveasfilename(defaultextension=".key",
                                     filetypes=[("Key", "*.key"), ("Todos", "*.*)"])
    if not p: return
    try:
        with open(p, "wb") as f: f.write(self._sym_key)
        self._log(self.sym_log, f"Clave guardada: {os.path.basename(p)}")
    except Exception as e: self._log(self.sym_log, f"Error: {e}")

def _sym_load_key(self):
    p = filedialog.askopenfilename(filetypes=[("Key", "*.key"), ("Todos", "*.*)"])
    if not p: return
    try:
        with open(p, "rb") as f: k = f.read().strip()
        Fernet(k); self._sym_key = k
        self._set_key_display(k.decode())
        self._log(self.sym_log, f"Clave cargada: {os.path.basename(p)}")
    except Exception as e: self._log(self.sym_log, f"Clave inválida: {e}")

def _sym_pick(self):
    p = filedialog.askopenfilename()
    if p: self._sym_file = p; self.sym_file_lbl.configure(text=os.path.basename(p))

def _get_sym_key(self):
    if self._sym_key: return self._sym_key
    raw = self.sym_key_box.get("1.0", "end").strip()
    if not raw: return None
    try: Fernet(raw.encode()); self._sym_key = raw.encode(); return self._sym_key
    except Exception: return None

def _sym_enc(self):
    k = self._get_sym_key()
    if not k: messagebox.showwarning("Aviso", "Genera o carga una clave primero."); return
    if not self._sym_file: messagebox.showwarning("Aviso", "Selecciona un archivo."); return
    try:
        with open(self._sym_file, "rb") as f: data = f.read()
        enc = Fernet(k).encrypt(data)
        out = self._sym_file + ".enc"
        with open(out, "wb") as f: f.write(enc)
        preview = base64.b64encode(enc[:80]).decode()

```

```

        self._log(self.sym_log, f"Cifrado: {os.path.basename(out)} ({len(enc)} bytes)")
        self._log(self.sym_log, f"Contenido cifrado (preview): {preview}")
    except Exception as e: self._log(self.sym_log, f"Error al cifrar: {e}")

def _sym_dec(self):
    k = self._get_sym_key()
    if not k: messagebox.showwarning("Aviso", "Genera o carga una clave primero."); return
    if not self._sym_file: messagebox.showwarning("Aviso", "Selecciona el archivo .enc."); return
    try:
        with open(self._sym_file, "rb") as f: data = f.read()
        plain = Fernet(k).decrypt(data)
        base = self._sym_file[:-4] if self._sym_file.endswith(".enc") else self._sym_file
        ext = os.path.splitext(base)[1]
        out = base.replace(ext, "_dec" + ext) if ext else base + "_dec"
        with open(out, "wb") as f: f.write(plain)
        text_preview = plain.decode("utf-8")
        self._log(self.sym_log, f"Descifrado: {os.path.basename(out)} ({len(plain)} bytes)")
        self._log(self.sym_log, f"Contenido recuperado: {text_preview}")

    except Exception as e: self._log(self.sym_log, f"Error al descifrar: {e}")

def _rsa_gen(self):
    try:
        self._rsa_priv = rsa.generate_private_key(public_exponent=65537, key_size=2048)
        self._rsa_pub = self._rsa_priv.public_key()
        self._rsa_upd(); self._log(self.rsa_log, "Par RSA-2048 generado.")
    except Exception as e: self._log(self.rsa_log, f"Error: {e}")

def _rsa_upd(self):
    p = "pública OK" if self._rsa_pub else "sin clave pública"
    v = "privada OK" if self._rsa_priv else "sin clave privada"
    self.rsa_status.configure(text=f"{p} | {v}", text_color=TEXT_DIM)

def _rsa_save(self, key_obj, get_bytes_fn, label):
    if not key_obj: messagebox.showwarning("Aviso", "Genera un par primero."); return
    p = filedialog.asksaveasfilename(defaultextension=".pem",
                                     filetypes=[("PEM", "*.pem"), ("Todos", "*.*)"])
    if not p: return
    try:
        with open(p, "wb") as f: f.write(get_bytes_fn(key_obj))
        self._log(self.rsa_log, f"Clave {label} guardada: {os.path.basename(p)}")
    except Exception as e: self._log(self.rsa_log, f"Error: {e}")

def _rsa_save_pub(self):
    self._rsa_save(self._rsa_pub,
                   lambda k: k.public_bytes(serialization.Encoding.PEM,
                                             serialization.PublicFormat.SubjectPublicKeyInfo),
                   "pública")

def _rsa_save_priv(self):
    self._rsa_save(self._rsa_priv,
                   lambda k: k.private_bytes(serialization.Encoding.PEM,
                                             serialization.PrivateFormat.PKCS8,
                                             serialization.NoEncryption()),
                   "privada")

```

```

def _rsa_load(self, parse_fn, attr, label):
    p = filedialog.askopenfilename(filetypes=[("PEM", "*.pem"), ("Todos", "*.*")])
    if not p: return
    try:
        with open(p, "rb") as f: data = f.read()
        setattr(self, attr, parse_fn(data))
        self._log(self.rsa_log, f"Clave {label} cargada: {os.path.basename(p)}")
        self._rsa_upd()
    except Exception as e: self._log(self.rsa_log, f"Error: {e}")

def _rsa_load_pub(self):
    self._rsa_load(serialization.load_pem_public_key, "_rsa_pub", "pública")

def _rsa_load_priv(self):
    self._rsa_load(lambda d: serialization.load_pem_private_key(d, password=None),
        "_rsa_priv", "privada")

def _rsa_pick(self):
    p = filedialog.askopenfilename()
    if p: self._rsa_file = p; self.rsa_file_lbl.configure(text=os.path.basename(p))

def _rsa_enc(self):
    if not self._rsa_pub: messagebox.showwarning("Aviso", "Carga la clave pública primero.");
return
    if not self._rsa_file: messagebox.showwarning("Aviso", "Selecciona un archivo."); return
    try:
        sym = Fernet.generate_key()
        with open(self._rsa_file, "rb") as f: data = f.read()
        enc_data = Fernet(sym).encrypt(data)
        enc_sym = self._rsa_pub.encrypt(
            sym, padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
                algorithm=hashes.SHA256(), label=None))
        out = self._rsa_file + ".rsa"
        with open(out, "wb") as f:
            f.write(len(enc_sym).to_bytes(4, "big"))
            f.write(enc_sym)
            f.write(enc_data)
        self._log(self.rsa_log, f"Cifrado (híbrido): {os.path.basename(out)} ({len(enc_data)}
bytes)")
        self._log(self.rsa_log, f"Contenido cifrado (preview): {preview}")
    except Exception as e: self._log(self.rsa_log, f"Error al cifrar: {e}")

def _rsa_dec(self):
    if not self._rsa_priv: messagebox.showwarning("Aviso", "Carga la clave privada primero.");
return
    if not self._rsa_file: messagebox.showwarning("Aviso", "Selecciona el archivo .rsa."); return
    try:
        with open(self._rsa_file, "rb") as f: raw = f.read()
        klen = int.from_bytes(raw[:4], "big")
        enc_sym = raw[4:4 + klen]; enc_data = raw[4 + klen:]
        sym = self._rsa_priv.decrypt(
            enc_sym, padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
                algorithm=hashes.SHA256(), label=None))
        plain = Fernet(sym).decrypt(enc_data)
        base = self._rsa_file[:-4] if self._rsa_file.endswith(".rsa") else self._rsa_file
        ext = os.path.splitext(base)[1]

```

```

        out = base.replace(ext, "_dec" + ext) if ext else base + "_dec"
        with open(out, "wb") as f: f.write(plain)
        text_preview = plain.decode("utf-8")
    except Exception as e: self._log(self.rsa_log, f"Error al descifrar: {e}")
    text_preview = str(plain[:100])

def refresh(self): pass

class AcademicApp(ctl.CTk):
    def __init__(self):
        super().__init__()
        self.title(APP_TITLE)
        self.geometry("1280x760"); self.minsize(1180, 700)
        self.configure(fg_color=BG_DARK)
        self.data = DataModel()
        self.sidebar = None; self.pages = {}; self.current_page = None
        self._username = "Usuario"
        self.login = LoginFrame(self, self.build_main)

    def build_main(self, username="Usuario"):
        self._username = username
        self.sidebar = Sidebar(self, self.show_page, self.logout, username)
        self.sidebar.pack(side="left", fill="y")
        self.content = ctl.CTkFrame(self, fg_color=BG_DARK, corner_radius=0)
        self.content.pack(side="right", fill="both", expand=True)
        for name, cls in [("Dashboard", DashboardPage),
                        ("Asignaturas", AsignaturasPage),
                        ("Categorías", CategoríasPage),
                        ("Estudiantes", EstudiantesPage),
                        ("Notas", NotesPage),
                        ("Cifrado", CifradoPage)]:
            self.pages[name] = cls(self.content, self)
        for page in self.pages.values():
            page.place(relx=0, rely=0, relwidth=1, relheight=1)
        self.show_page("Dashboard")

    def show_page(self, name):
        self.current_page = name
        if self.sidebar: self.sidebar.set_active(name)
        page = self.pages.get(name)
        if page:
            page.lift()
            if hasattr(page, "refresh"): page.refresh()

    def refresh_all(self):
        for page in self.pages.values():
            if hasattr(page, "refresh"): page.refresh()

    def logout(self):
        if not messagebox.askyesno("Cerrar sesión", "¿Deseas cerrar sesión?"): return
        for page in self.pages.values(): page.destroy()
        if self.sidebar: self.sidebar.destroy()
        self.sidebar = None; self.pages = {}; self.current_page = None
        self._username = "Usuario"
        self.login = LoginFrame(self, self.build_main)

```

```
if __name__ == "__main__":  
    AcademicApp().mainloop()
```



sistemaUniversidadT
ec.py